

Homework 2 - Sorting

Name: 임도협 (Dohyeop Lim)

StudentID: 24101518

[1. Insertion Sort]

A. Implement `insertion_sort(...)` in `sort.c`. The function takes three arguments: an array of integers, `low` and `high` indices to denote the range to be sorted. Thus, the function sorts the array of range `[low, high)`. The function must produce the array of integers, whose elements are located in ascending order.

a

[2. Merge Sort]

A. Implement `merge(...)` in `sort.c`. The function takes four arguments: an array of integers, `low`, `mid`, and `high` indices. The `low` and `high` indices denote the range to be sorted. The `mid` is defined by $(low + high) / 2$. Therefore, the `low` and the `mid` denote the start indices of the left and the right subarrays, respectively. The output of the function must be the sorted array of size `high - low`.

a

B. Implement `merge_sort(...)` in `sort.c`.

a

[3. Quick Sort]

A. Implement `partition(...)` in `sort.c`. The function takes three arguments: an array of integers, `low`, and `high` indices. The `low` and `high` indices denote the range to be partitioned. The function outputs the index of pivot in the partitioned array. For the partitioning, you should select random pivot. Also, you can use `swap()` in `util.h`.

a

B. Implement `quick_sort(...)` in `sort.c`.

a

[3. Comparison of Sorting Algorithms]

- A. Explain the difference between three types of input arrays in `main.c`: `random_array`, `partial_random_array1`, `partial_random_array2`.

a

- B. Measure the execution time of three algorithms using input arrays of different size: 10, 30, 50, 100, 500, 1000, 10000, 100000, 1000000, 10000000. Plot graph to compare the execution time of three algorithm for each type of random array.

a

- C. Let's consider `random_array` data. Which algorithm is the best when the array size is small (e.g., 10, 30, 50). Explain why.

a

- D. Let's consider `random_array` data. Which algorithm becomes the eventually winner? Explain why.

a

- E. Let's consider the insertion sort algorithm. Which types of input arrays does insert sort have an advantage over other algorithms? Explain why.

a

- F. Explain why quick sort is faster than merge sort.

a

- G. Explain the advantages that merge sort has over quick sort.

a

[8. Faster Merge Sort]

- A. Implement the fast merge sort algorithm. The improved version divides the input array only until the size of the subarrays becomes less than 32. It then perform the insertion sort to conquer these small subarrays.

a

- B. Compare the fast merge sort with its original one on the settings of problem 4.B. Is performance improvement observable? Explain why it is faster.

a

- C. Implement the faster merge sort algorithm by modifying the fast merge sort. This version first determine the necessity of merge operation before merging, and then skips the merge operation, when it is unnecessary.

a

- D. For which types of input array will this version produce performance improvements?. Explain why.

a